УДК 004.334
DOI: 10.21661/r-81019

П.Е. Соколов, А.С. Бутов, И.С. Пеплов

Методика разработки RTL асинхронного блока FIFO, оптимизированного по площади и потреблению

Аннотация

В данной статье представлено несколько новых проектов асинхронных буферов FIFO. В то время как многие из существующих буферов FIFO имеют высокую пропускную способность при высокой латентности, цель авторов заключается в достижении низкой латентности, сохраняя хорошую пропускную способность. Архитектуры в данной статье основаны на круговых массивах памяти, ячейки которых подключены к общим шинам данных. Данные не перемещаются в памяти после их включения в очередь. Чтение и запись из каждой ячейки контролируется двумя охватывающими весь массив памяти по кругу указателями: один указатель позволяет включать данные в очередь, а другой – извлекать из неё. С того момента, когда были найдены проблемы в природе кода Грея, обе архитектуры, представленные в этой статье, используют модифицированные счетчики кодов Грея для сравнения указателей и адресации массива памяти. В первой архитектуре представленного буфера FIFO массив памяти адресуется двоичным кодом в то время как указатели конвертируются в код Грея и сравниваются синхронно. Во второй архитектуре FIFO массив памяти адресуется кодами Грея, но сравнение указателей происходит асинхронно. Эти изменения дают преимущество над первой архитектурой в снижении латентности и потребляемой мощности.

Ключевые слова: асинхронные FIFO, модифицированные счетчики, код Грея, обобщение, сравнение указателей.

P.E. Sokolov, A.S. Butov, I.S. Peplov

RTL design techniques of asynchronous FIFO memory blocks with area and power consumption optimizations

Abstract

This paper presents several new asynchronous FIFO designs. While most existing FIFO's have higher throughput for higher latency, our goal is to achieve very low latency while maintaining good throughput. The designs are implemented as circular arrays of cells connected to common data buses. Data items are not moved around the array once; they are enqueued. Each cell's input and output behavior is dictated by the flow of two tokens around the ring: one that allows enqueuing data and one that allows dequeuing data. Since the problems were found in gray code's nature, both FIFO architectures represented in this paper use a modified gray code counters to pointers comparison and addressing the memory array. In the first architecture of FIFO buffer, memory array is addressed by binary code while the pointers are converted to gray codes and compared synchronously. In the second architecture of FIFO buffer memory array is addressed by gray code, but the gray code pointers comparison proceeds asynchronously. These changes give the advantage over the first architecture to reduce latency and power consumption.

Keywords: asynchronous FIFO, Gray count modification, synthesis, pointers comparison.

Современные отечественные высокопроизводительные СБИС и СнК (системы на кристалле) требуют наличия надёжных быстродействующих средств обмена данными с периферийными устройствами и блоками, которые работают на других, порой более низких частотах, чем сама СБИС или СнК.

Подобными и популярными на сегодняшний день средствами являются буферы памяти типа FIFO («First In, First Out»).

В электронных системах, такие буферы рекомендуется устанавливать между устройствами, работающими на разных скоростях или нерегулярно. В противном случае, самый медленный компонент определяет скорость работы остальных, вовлеченных в обмен данными. Такими электронными системами могут являться связки высокопроизводительных процессоров и периферийных блоков сбора и обработки информации.

Принцип работы FIFO заключается в последовательной загрузке данных внутрь буфера на одной частоте и последовательной выгрузке их на другой частоте, как правило, превышающей частоту загрузки. Первое, записанное в буфер, слово будет прочитано первым.

В данной статье представлены два варианта архитектурной реализации буфера FIFO. Обозначаются они как FIFO1 и FIFO2, и имеют ряд отличий, подробное описание которых приведено ниже.

В основе архитектур FIFO1 и FIFO2 (рис. 1. и рис. 2 соотв.), лежит кольцевой буфер с двумя счётчиками [1, с. 1]. Один из этих счётчиков синхронизирован с тактовым сигналом write_clk, ответственен за установку флага «полон» (далее full) и формирует указатель записи wr_addr, который адресует данные, записываемые в FIFO. Другой счётчик синхронизирован с тактовым сигналом read_clk, ответственен за установку флага «пуст» (далее empty) и формирует указатель чтения rd_addr, адресующий данные, читаемые из FIFO.

Трудность, заключающаяся в проектировании такого типа FIFO, связана с генерацией указателей чтения и записи и поиском надёжного способа определения состояния full и empty.

В архитектуре FIFO1 для определения состояния empty указатель записи wr_addr транслируется в код Грэя (wr_addr_gray) и с помощью цепи синхронизации передаётся из домена, тактируемого сигналом write_clk, в домен, который тактируется уже сигналом read_clk, для сравнения с указателем rd_addr. Если rd_addr равен синхронизированному wr_addr_gray, то FIFO немедленно перейдёт в состояние empty. Снятие же флага empty происходит с задержкой, равной времени синхронизации сигнала wr_addr_gray из домена write_clk в домен read_clk.

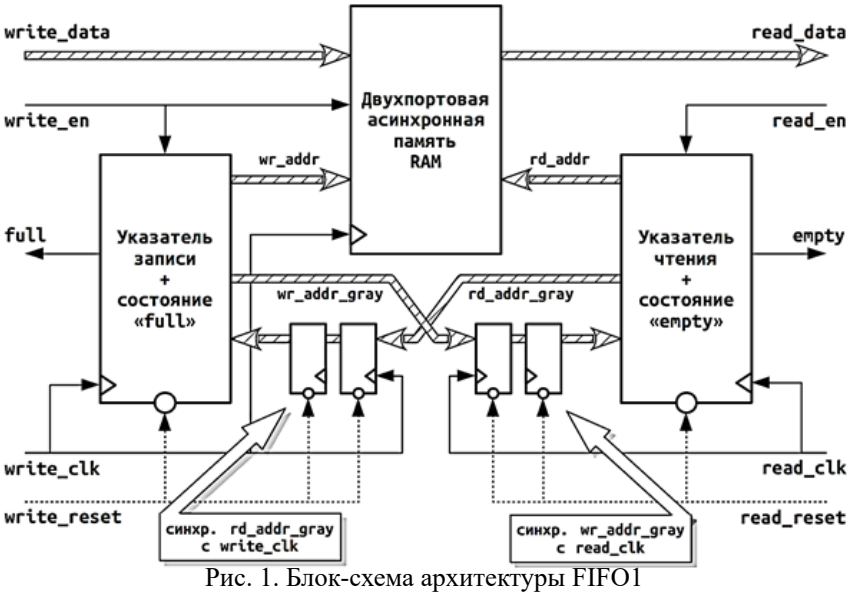

Рис. 1. Блок-схема архитектуры FIFO1

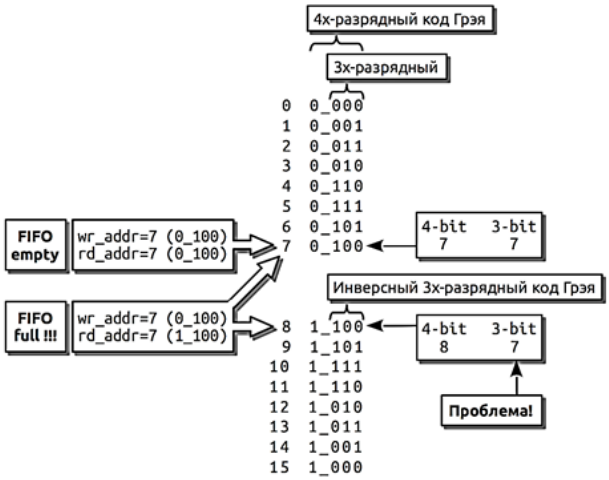

Рис. 2. Проблема извлечения 3х-разрядного кода Грэя из 4х-разрядного

Аналогично, для определения состояния full, указатель чтения rd_addr транслируется в код Грэя (rd_addr_gray) и с помощью цепи синхронизации передаётся из домена, тактируемого сигналом read_clk, в домен, который тактируется уже сигналом write_clk, для сравнения с указателем wr_addr. Если wr_addr оказался непосредственно позади синхронизированного rd_addr_gray, то FIFO немедленно перейдёт в состояние full. Снятие же флага full происходит с задержкой, равной времени синхронизации сигнала rd_addr_gray из домена write_clk в домен read_clk.

Для правильного определения состояний full и empty указатели чтения и записи должны быть на один разряд шире, чем ширина адреса памяти. Это нужно для того, чтобы определить какой из указателей идёт впереди, а какой позади другого.

Состояние full, в отличие от empty, определить не так просто. Использовать обычный счётчик Грэя, чей выход на один разряд шире адреса, здесь не получится. Проблема заключается в том, что код Грэя симметричен за исключением старшего разряда (см. рис. 2).

На рис. 2 данная проблема показана на примере получения 3-разрядного кода Грэя из 4-разрядного. В этом примере 3-разрядный код Грэя используется для адресации памяти, а дополнительный старший разряд 4-разрядного кода используется для определения состояний full и empty. Если в FIFO сперва были записаны строки от 0 до 6, а затем эти же строки были прочитаны, то FIFO перейдёт в состояние empty, оба указателя станут равными и укажут на адрес с номером 7. При следующей операции записи, 4х-разрядный указатель записи будет увеличен на 1 и его старший разряд перестанет быть равным MSB указателя чтения, но их оставшиеся три разряда останутся равными, что повлечёт переход FIFO в ложное состояние full и перезапись ячейки памяти.

Для корректной адресации массива памяти и исключения ложных состояний, в FIFO1 и FIFO2 используется модифицированный код Грэя (рис. 3).

В этом случае, при изменении первого MSB в коде Грэя, инвертируется второй по счёту MSB. В итоге, получаемый из n-разрядного, (n - 1)-разрядный код Грэя больше не является симметричным. При этом, будучи модифицированным, данный код перестанет быть «честным» кодом Грэя, так как при определённых переключениях будут меняться уже два разряда, а не один.

В архитектуре FIFO1 используется счётчик кодов Грэя, включающий в себя два набора регистров (см. рис. 4 счетчик Грэя). В данной реализации счётчика отсутствует необходимость в трансляции кодов Грэя обратно в двоичный код. Набор регистров, отвечающих за хранение двоичных значений счётчика, используются для адресации массива памяти FIFO.

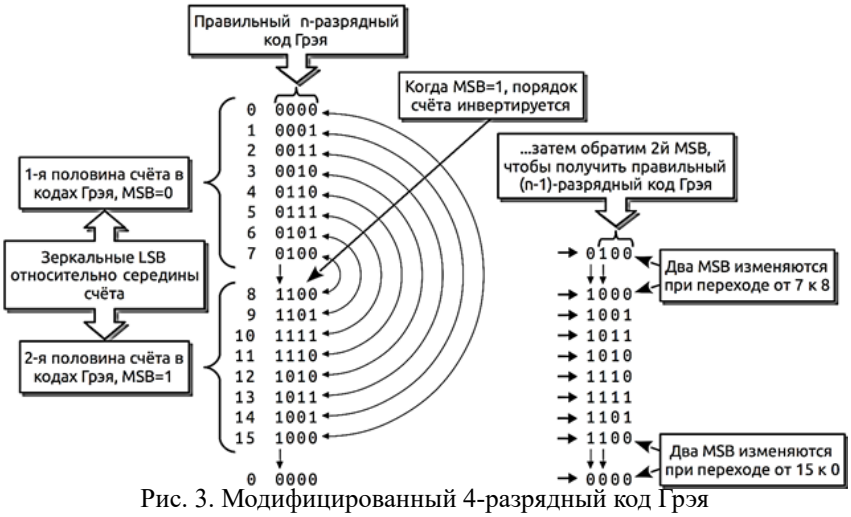

Рис. 3. Модифицированный 4-разрядный код Грэя для извлечения из него правильного 3-разрядного кода

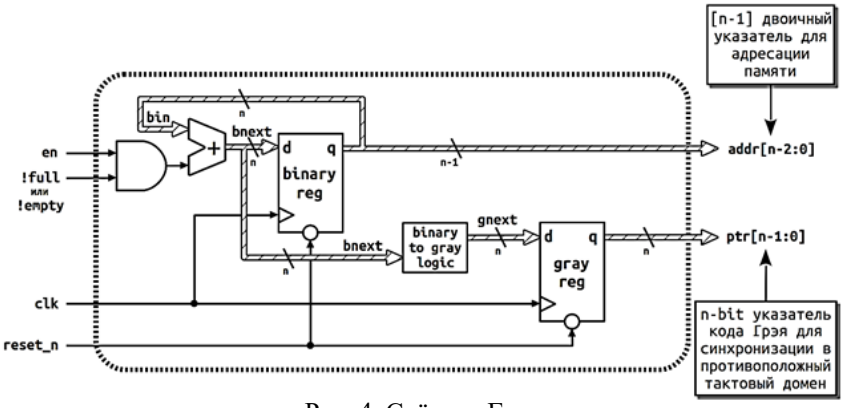

Рис. 4. Счётчик Грэя

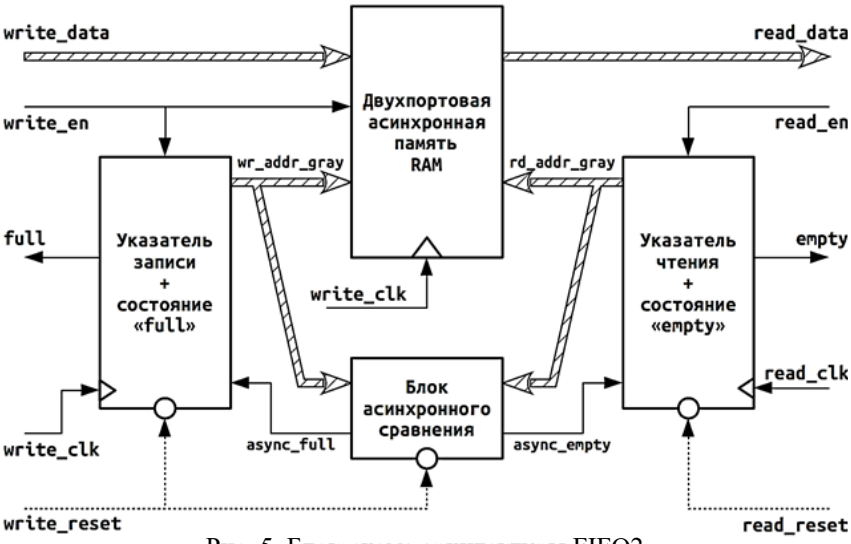

Рис. 5. Блок-схема архитектуры FIFO2

В архитектуре FIFO2 (рис. 5), в отличие от FIFO1 (рис. 1), внутренний массив памяти адресуется кодами Грэя. Счётчик кодов Грэя в этом случае выглядит следующим образом (рис. 6).

За определение состояний full и empty отвечает блок асинхронного сравнения [2, с. 2]. Адресное пространство указателей записи и чтения делится на четыре квадранта (рис. 7 и рис. 8). Это позволяет определить, что либо указатель записи догоняет указатель чтения, либо, наоборот, указатель чтения догоняет указатель записи.

Согласно схеме на рис. 7, если указатель записи находится на один квадрант позади указателя чтения, то это приведёт к установке 1 на выходе защёлки (latch) в блоке асинхронного сравнения и означает, что FIFO возможно скоро заполнится. В случае на рис. 8, если указатель чтения находится на один квадрант позади указателя записи, то это приведёт к сбросу защёлки в 0 и означает, что FIFO возможно скоро опустеет.

Было проведено функциональное тестирование блоков FIFO1 и FIFO2, синтезированных по нормам 45 нм, в трёх режимах работы (рис. 9):

– в первом режиме производилась запись одного слова данных, а затем его считывание;

– во втором режиме производилась пакетная запись «шахматного» кода до заполнения массива памяти и установки флага full, затем подавалась команда на считывание записанных данных из FIFO до установки флага empty;

– в третьем режиме производилась пакетная запись случайных комбинаций кода до заполнения массива памяти и установки флага full, затем подавалась команда на считывание записанных данных из FIFO до установки флага empty.

Результаты логического синтеза с расчётом потребляемой мощности рассматриваемых архитектур FIFO с учётом переключательной активности их вентилей в среде САПР «Cadence» по нормам 45 нм следующие (таблица 1).

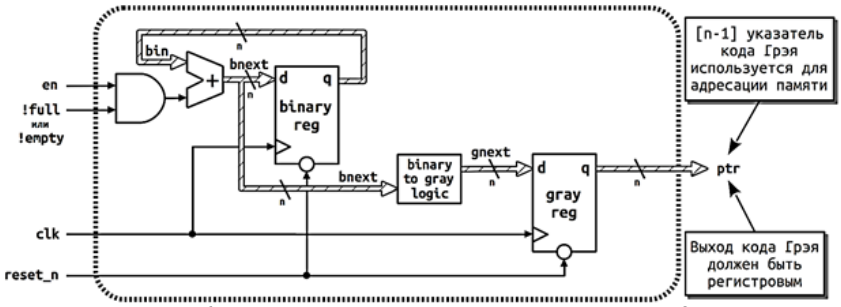

Рис. 6. Счётчик кодов Грэя в архитектуре FIFO2

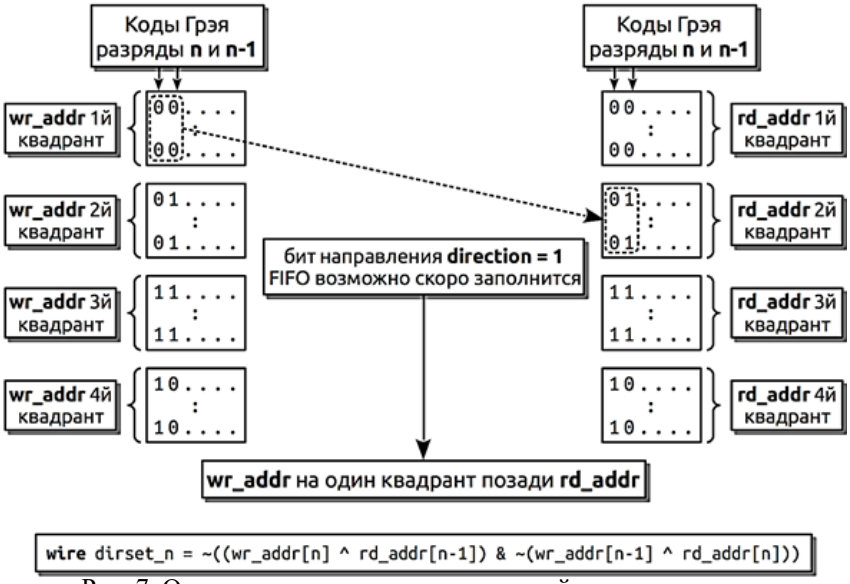

Рис. 7. Определение положения указателей чтения и записи, а также прогнозирование состояния full

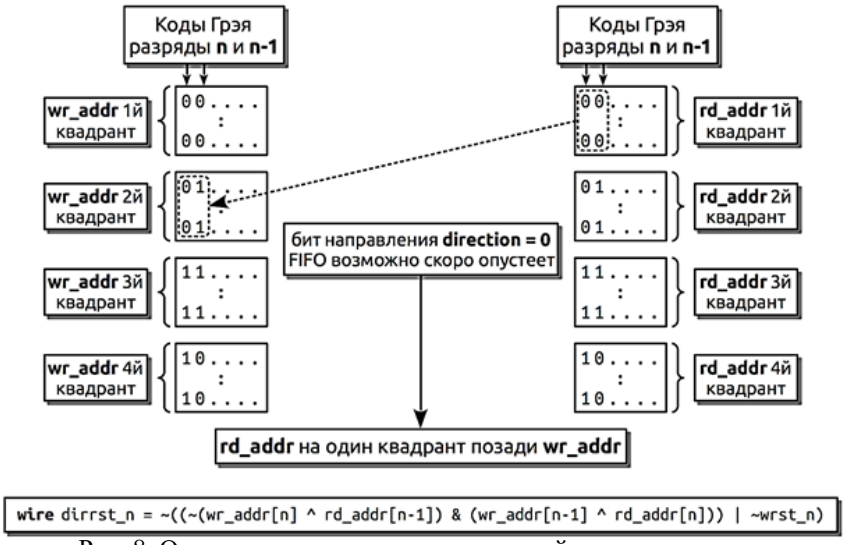

Рис. 8. Определение положения указателей чтения и записи, а также прогнозирование состояния empty

Таблица 1

Результаты логического синтеза FIFO1 и FIFO2

	Area, um 2		AVG Power, mW	
	cell	total	leakage	total
FIFO1	9904	24245	0.099	0.982
FIFO2	9776	24033	0.098	0.976

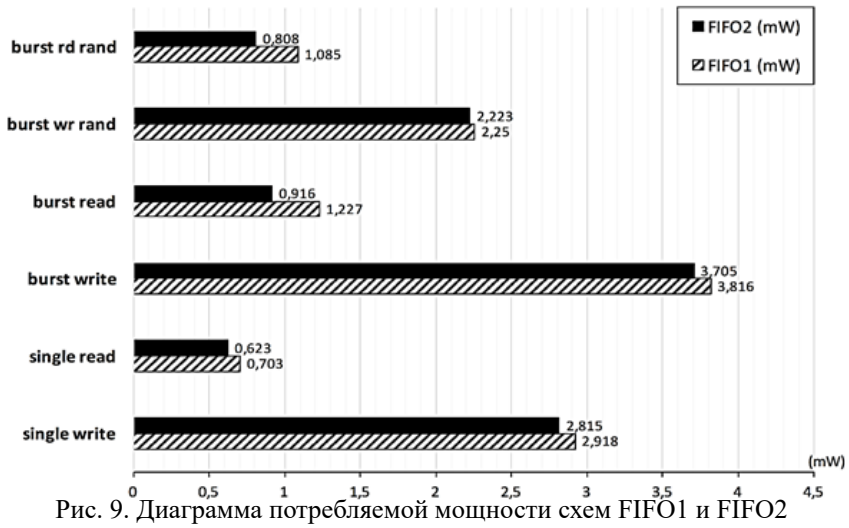

Рис. 9. Диаграмма потребляемой мощности схем FIFO1 и FIFO2 в ходе функционального тестирования (меньше – лучше)

По результатам разработки и исследования двух блоков блоков FIFO с асинхронной архитектурой было выявлены преимущества архитектуры FIFO2 перед FIFO1:

– адресация массива памяти кодами Грэя позволила упростить архитектуру счётчиков указателей записи и чтения;

– асинхронное сравнение указателей позволило снизить потребляемую мощность до 25%.

Данный вывод подтверждают результаты логического синтеза блоков FIFO1 и FIFO2 и полученные данные по потребляемой мощности в различных режимах работы, представленные на графиках выше.

Литература

1. Chelcea T. Low-latency asynchronous FIFO's using token rings / T. Chelcea, S.M. Nowick. – New York: Columbia University, 2000. – P. 1–2.
2. Wang X. A RTL Asynchronous FIFO Design Using Modified Micropipeline / X. Wang, J. Nurmi. – Tampere: Tampere University of Technology, 2006. – P. 2–3.